Lecture 4: Loss Function, Cost Function and Activation Function

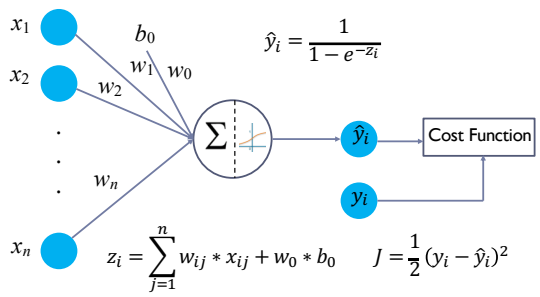
Yaxin Bi
School of Computing
University of Ulster

1

Contents

2

A Scenario of Cost Function



3

Cost Function

- A cost function is a measure of error between what value your model predicts and what the value actually is. For example, say we wish to predict the value $\hat{y}_i$ for data point x_i .

$$\hat{y}_i(z_w(x_i, w_i))$$

represent the prediction or output of some arbitrary model for the point x_i with parameters w. One of the many cost functions could be

$$J = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)_w^2$$

4

L_2 and L_1 Loss Functions

- L_2 loss is to minimize the error which is the sum of the all the squared differences between the true value and the predicted value through training to find the w.

$$J = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|_w$$

- L_1 loss is to minimize the error which is the sum of the all the absolute differences between the true value and the predicted value through training to find the w.

5

Activation Function

- An activation function transforms the outputs of nodes to the shape/representation.
  - A simple example could be max(0, x_i), a function which outputs 0 if the input x_i is negative or x_i if the input x_i is positive. This function is known as the "ReLU" or "Rectified Linear Unit" activation function.
- The choice of which function(s) are best for a specific problem using a particular neural architecture is still under a lot of discussions.
- However, these representations are essential for making high-dimensional data (linearly) separable, which is always used in neural networks.

6

Difference between Cost Function and Loss Function?

- In the literature, cost and loss functions are synonymous (this function is often called as error function).
- The more general scenario is to define an objective function first, and then optimize it. This objective function could be to
  - maximize the posterior probabilities (e.g., naive Bayes)
  - maximize a fitness function (genetic programming)
  - maximize the total reward/value function (reinforcement learning)
  - maximize information gain/minimize child node impurities (CART decision tree classification)

7

7

Difference between Cost Function and Loss Function? (cont'd)

- minimize a mean squared error cost (or loss) function (CART, decision tree regression, linear regression, adaptive linear neurons, …
- maximize log-likelihood or minimize cross-entropy loss (or cost) function
- minimize hinge loss (support vector machine)
- The loss function (or error) is for **a single training example**, while the cost function is over the **entire training set** (or mini-batch for mini-batch gradient descent).

8

8

Loss Function

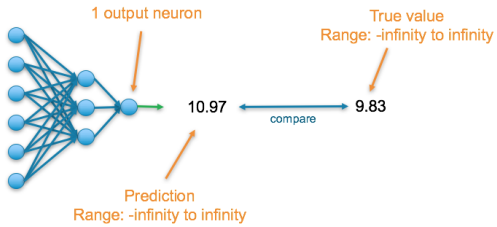
- **Loss function** is a method of evaluating "how well your algorithm models over your dataset".
- If your predictions are totally off, your loss function will output a higher number.
- If they're pretty good, it'll output a lower number.
- As you tune your algorithm to try and improve your model, your loss function will tell you if you're improving or not.
- 'Loss' helps us to understand how much the predicted value differ from actual value

9

9

Regression: Predicting a Numerical Value

- The final layer of the neural network must have one neuron and the value it returns is a continuous numerical value.

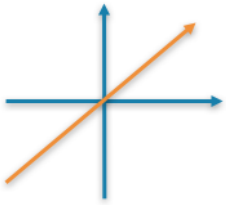


10

10

Final Activation Function

- **Linear** — This results in a numerical value which we require



$$f(z) = az$$

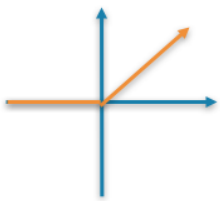
Range: -infinity to infinity

11

11

Final Activation Function (cont'd)

- **ReLU** — This results in a numerical value greater than 0



$$relu(z) = \max(0, z)$$

Range: 0 to infinity

12

12

Loss Functions

- **Mean squared error (MSE)** — This finds the average squared difference between the predicted value and the true value

$$MSE = \frac{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}{m}$$

(where y_i is the truth values and $\hat{y}_i$ is the predicted value)

- **Mean Absolute Error (MAE)** — This finds the average absolute difference between the predicted value and the true value

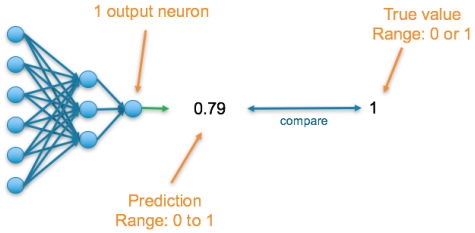
$$MAE = \frac{\sum_{i=1}^{m}|y_i - \hat{y}_i|}{m}$$

13

13

Classification: Predicting a binary outcome

- The final layer of the neural network will have one neuron and will return a value between 0 and 1, which can be inferred as a probably.

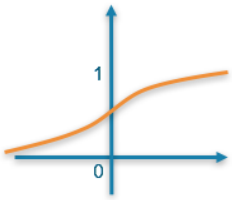


14

14

Final Activation Function

- **Sigmoid** — This results in a value between 0 and 1 which we can infer to be how confident the model is of the example being in the class



$$sigmoid(z) = \frac{1}{1 + e^{-z}}$$

Range: 0 to 1

15

15

Loss Function

- **Binary Cross Entropy** — Cross entropy quantifies the difference between two probability distribution. Our model predicts a model distribution of $\{\hat{y}, 1\text{-}\hat{y}\}$ as we have a binary distribution. We use binary cross-entropy to compare this with the true distribution $\{y, 1\text{-}y\}$

$$Binary\ Cross\ Entropy = -\Big(\big(y \log \hat{y}\big) + (1-y)\log(1-\hat{y})\Big)$$

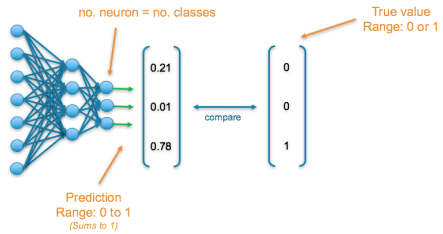
- Notice that when actual label is 1 ($y = 1$), second half of function disappears whereas in case actual label is 0 ($y = 0$) first half is dropped off. We are just multiplying the log of the actual predicted probability for the ground truth class. An important aspect of this is that cross entropy loss penalizes heavily the predictions that are *confident but wrong*.

16

16

Classification: Predicting a single label from multiple classes

- The final layer of the neural network will have one neuron for each of the classes. The output then results in a probability distribution as it sums to 1.

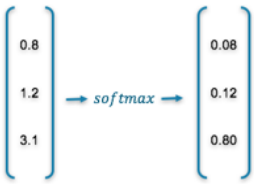


17

17

Final Activation Function

- **Softmax** — This results in values between 0 and 1 for each of the outputs which all sum up to 1. Consequently, this can be inferred as a probability distribution



$$softmax(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Range: 0 to 1

Divides output so that the total sum of the output is equal to 1

18

18

Loss Function

- **Cross Entropy** — Cross entropy quantifies the difference between two probability distribution. Our model predicts a model distribution of $\{\hat{y}_1, \hat{y}_2, \hat{y}_3\}$ (where $\hat{y}_1 + \hat{y}_2 + \hat{y}_3 = 1$). We use cross-entropy to compare this with the true distribution $\{y_1, y_2, y_3\}$

$$Cross\ Entropy = -\sum_{i=1}^{k} y_i \log \hat{y}_i$$

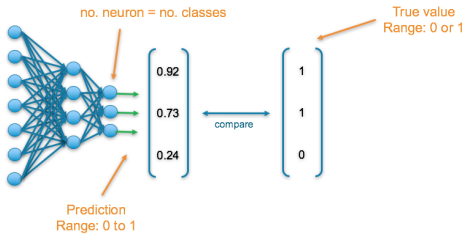
where k is the number of classes.

19

19

Classification: Predicting multiple labels from multiple classes

- The final layer of the neural network will have one neuron for each of the classes and they will return a value between 0 and 1, which can be inferred as a probably.

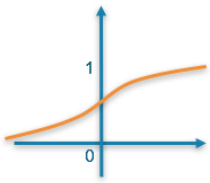


20

20

Final Activation Function

- **Sigmoid** — This results in a value between 0 and 1 which we can infer to be how confident it is of it being in the class



$$sigmoid(z) = \frac{1}{1 + e^{-z}}$$

Range: 0 to 1

21

21

Loss Function

- **Binary Cross Entropy** — Cross entropy quantifies the difference between two probability distribution. Our model predicts a model distribution of $\{\hat{y}, 1\text{-}\hat{y}\}$ (binary distribution) for each of the classes. We use binary cross-entropy to compare these with the true distributions $\{y, 1\text{-}y\}$ for each class and sum up their results

$$Binary\ Cross\ Entropy = -\sum_{i=1}^{k} \left(\left(y_i \log \hat{y}_i \right) + (1 - y_i) \log(1 - \hat{y}_i) \right)$$

22

22

Summary Table

- The following table summarizes the above information to allow you to quickly find the final layer activation function and loss function that is appropriate to your use-case

Problem Type	Output Type	Final Activation Function	Loss Function
Regression	Numerical value	Linear	Mean Squared Error (MSE)
Classification	Binary outcome	Sigmoid	Binary Cross Entropy
Classification	Single label, multiple classes	Softmax	Cross Entropy
Classification	Multiple labels, multiple classes	Sigmoid	Binary Cross Entropy

23

23

Regularization Terms

- Generally cost and loss functions are synonymous but cost function can contain regularization terms in addition to loss function although it is not always necessary.

Given a n-order polynormal function

$$y_w(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_n x^n$$

Regularization terms

$$J(w) = \frac{1}{2m}\left[\sum_{i=1}^{m} \left(w_i x^i - y^i\right)^2 + \lambda \sum_{i=1}^{m} w_i^2\right]$$

The regularization parameter λ is a control on your fitting parameters

24

24

Regularization Terms (cont'd)

- As the magnitudes of the fitting parameters increase, there will be an increasing penalty on the cost function. This penalty is dependent on the squares of the parameters as well as the magnitude of λ.
- Here's the regularized cross-entropy:

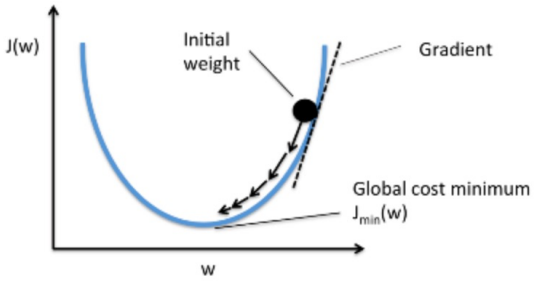
$$J(w) = -\frac{1}{m}\sum_{j=1}^{m}\left[y_j \log(\hat{y}_j) + (1-y_j)\log(1-\hat{y}_j)\right] + \frac{\lambda}{m}\sum_{j=1}^{m}(w_j)^2$$

The first term is just the usual expression for the cross-entropy. A second term, namely the sum of the squares of all the weights in the network. This is scaled by a factor λ/m, where $\lambda > 0$ is known as the *regularization parameter*

25

25

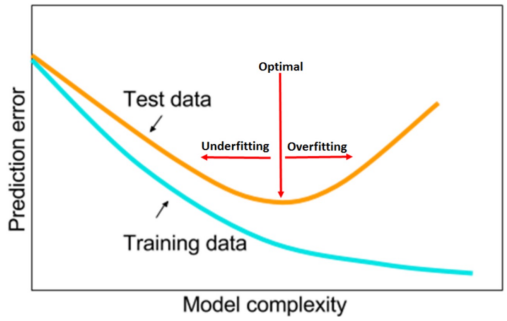
Cost against Weights



26

26

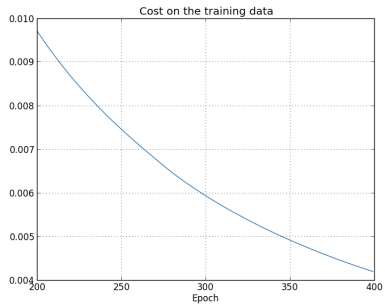
Optimal Capacity, Falling between Underfitting and Overfitting



27

27

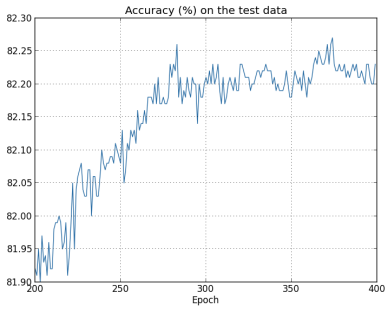
Evaluating Model Performance by Training



28

28

Evaluating Model Performance by Testing



29

29

Summary of Cost Function

- It is a function that **measures the performance of a Machine Learning model** for given data. Cost Function quantifies the error between predicted values and expected values.
- Depending on the problem Cost Function can be formed in many different ways. The purpose of Cost Function is to be either:
  - **minimized** — then returned value as **cost, loss** or **error**. The goal is to find the values of model parameters for which Cost Function return as small number as possible.
  - **maximized** — then the value it yields as a **reward**. The goal is to find values of model parameters for which returned number is as large as possible.

30

30

Activation Function Types

- Linear function
- Binary Step function
- Non-Linear function

31

31

Linear Types

- A linear activation function takes the form

$$y = Wx + b$$

where it takes the inputs (x_i's), multiplied by the weights(W_i's) for each neuron, and creates an output proportional to the input. In simple term, weighted sum input is proportional to output.

32

32

Binary Step Function

- Binary step function are popular known as "Threshold function". It is very simple function.
- The gradient (differential) of the binary step function is zero, which is the very big problem in backpropagation for updating weights.
- Another problem with a step function is that it can handle binary class problem alone. (Though with some tweak we can use it for multi-class problem)

33

33

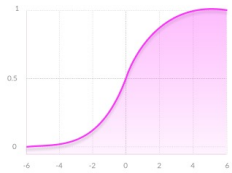
Non-linear Types

- Most modern neural network use the non-linear function as their activation function to fire the neuron.
  - Because they allow the model to create complex mappings between the network's inputs and outputs, which are essential for learning and modeling complex data, such as images, video, audio, and data sets which are non-linear or have high dimensionality.
  - Differential are possible in all the non-linear function.
  - Stacking of network is possible, which helps us in creating the deep neural nets.
- The Nonlinear Activation Functions are mainly divided on the basis of their **range or curves.**

34

34

Sigmoid: advantages



- **Smooth gradient**, preventing "jumps" in output values.
- **Output values bound** between 0 and 1, normalizing the output of each neuron.
- **Clear predictions** — For X above 2 or below -2, tends to bring the Y value (the prediction) to the edge of the curve, very close to 1 or 0. This enables clear predictions.

35

35

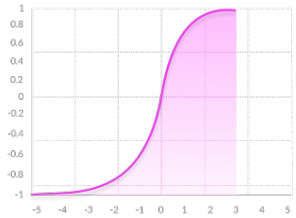
Sigmoid: disadvantages

- **Vanishing gradient** — for very high or very low values of X, there is almost no change to the prediction, causing a vanishing gradient problem. This can result in the network refusing to learn further or being too slow to reach an accurate prediction.
- **Outputs (y values) not zero centered**.
- **Computationally expensive**

36

36

Hyperbolic Tangent

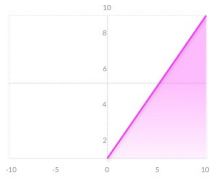


- **Zero centered** — making it easier to model inputs that have strongly negative, neutral, and strongly positive values.
- Otherwise like the Sigmoid function.

37

37

ReLU (Rectified Linear Unit)



- **Computationally efficient** — allows the network to converge very quickly
- **Non-linear —** although it looks like a linear function, ReLU has a derivative function and allows for backpropagation
- **The Dying ReLU problem** — when inputs approach zero, or are negative, the gradient of the function becomes zero, the network cannot perform backpropagation and cannot learn

38

38